

```

import java.io.*;
import java.time.LocalDateTime;
import java.util.*;

public class LibraryManagementSystem {
    private List<LibraryItem> items;
    private List<Member> members;
    private final Scanner scanner = new Scanner(System.in);
    private final Set<Integer> usedIds = new HashSet<>();

    public static void main(String[] args) {
        LibraryManagementSystem system = new LibraryManagementSystem();
        system.loginMenu();
    }

    public LibraryManagementSystem() {
        loadData();
    }

    public void loginMenu() {
        while (true) {
            System.out.println("\n--- Welcome to Library System ---");
            System.out.println("1. Log in");
            System.out.println("2. Exit");
            System.out.print("Choose option: ");
            int choice = scanner.nextInt();
            scanner.nextLine();

            switch (choice) {
                case 1 -> {
                    System.out.print("Enter username: ");
                    String user = scanner.nextLine();
                    System.out.print("Enter password: ");
                    String pass = scanner.nextLine();
                    if (user.equals("admin") && pass.equals("1212")) {
                        run();
                    } else {
                        System.out.println("Login failed.");
                    }
                }
                case 2 -> {
                    System.out.println("Exiting...");
                    return;
                }
                default -> System.out.println("Invalid choice.");
            }
        }
    }
}

```

```

public void run() {
    int choice;
    do {
        System.out.println("\n--- Library Menu ---");
        System.out.println("1. Add Book");
        System.out.println("2. Add Magazine");
        System.out.println("3. Register Member");
        System.out.println("4. Borrow Item");
        System.out.println("5. Return Item");
        System.out.println("6. Display Items");
        System.out.println("7. Display Members");
        System.out.println("8. Logout");
        System.out.print("Enter choice: ");
        choice = scanner.nextInt();

        switch (choice) {
            case 1 -> addItem(true);
            case 2 -> addItem(false);
            case 3 -> registerMember();
            case 4 -> borrowItem();
            case 5 -> returnItem();
            case 6 -> displayItems();
            case 7 -> displayMembers();
            case 8 -> System.out.println("Logging out...");
            default -> System.out.println("Invalid option.");
        }
    } while (choice != 8);
}

private void addItem(boolean isBook) {
    System.out.print("Enter unique ID: ");
    int id = scanner.nextInt();
    scanner.nextLine();

    System.out.print("Enter Title (case-sensitive): ");
    String title = scanner.nextLine();

    System.out.print("Quantity: ");
    int qty = scanner.nextInt();

    LibraryItem existing = findItemByIdAndTitle(id, title);
    if (existing != null) {
        existing.quantity += qty;
        log("Updated quantity for existing item: " + title + " (ID: " + id + ")");
        System.out.println("Item exists. Quantity updated.");
    } else {
        if (usedIds.contains(id)) {

```

```

        System.out.println("ID already used. Please use a unique ID.");
        return;
    }
    LibraryItem item = isBook ? new Book(id, title, qty) : new Magazine(id, title, qty);
    items.add(item);
    usedIds.add(id);
    log(("isBook ? \"Book\" : \"Magazine\"") + " added: " + title + " (ID: " + id + ")");
    System.out.println("Item added with ID: " + id);
}
saveData();
}

```

```

private void registerMember() {
    System.out.print("Enter unique ID: ");
    int id = scanner.nextInt();
    scanner.nextLine();
    if (usedIds.contains(id)) {
        System.out.println("ID already used. Please enter a different ID.");
        return;
    }
}

```

```

System.out.print("Name: ");
String name = scanner.nextLine();
System.out.print("Password: ");
String password = scanner.nextLine();

```

```

members.add(new Member(id, name, password));
usedIds.add(id);
log("Member registered: " + name + " (ID: " + id + ")");
saveData();
System.out.println("Member registered with ID: " + id);
}

```

```

private void borrowItem() {
    Member member = login();
    if (member == null) return;

    System.out.print("Enter item ID: ");
    int itemId = scanner.nextInt();
    LibraryItem item = findItemById(itemId);
    if (item == null) {
        System.out.println("Item not found.");
        return;
    }
}

```

```

try {
    member.borrowItem(item);
    saveData();
}

```

```

        System.out.println("Item borrowed successfully.");
    } catch (Exception e) {
        System.out.println("Borrow failed: " + e.getMessage());
    }
}

```

```

private void returnItem() {
    Member member = login();
    if (member == null) return;

    try {
        member.returnItem();
        saveData();
        System.out.println("Item returned successfully.");
    } catch (Exception e) {
        System.out.println("Return failed: " + e.getMessage());
    }
}

```

```

private Member login() {
    System.out.print("Member ID: ");
    int id = scanner.nextInt();
    scanner.nextLine();
    System.out.print("Password: ");
    String pw = scanner.nextLine();
    for (Member m : members) {
        if (m.getId() == id && m.authenticate(pw)) return m;
    }
    System.out.println("Login failed.");
    return null;
}

```

```

private void displayItems() {
    if (items.isEmpty()) {
        System.out.println("No items to display.");
        return;
    }
    for (LibraryItem item : items) {
        item.displayDetails();
    }
}

```

```

private void displayMembers() {
    if (members.isEmpty()) {
        System.out.println("No members to display.");
        return;
    }
    for (Member m : members) {

```

```

        System.out.println("ID: " + m.getId() + ", Name: " + m.getName());
        m.displayBorrowed();
    }
}

private LibraryItem findItemById(int id) {
    for (LibraryItem i : items) {
        if (i.getId() == id) return i;
    }
    return null;
}

private LibraryItem findItemByIdAndTitle(int id, String title) {
    for (LibraryItem i : items) {
        if (i.getId() == id && i.getTitle().equals(title)) return i;
    }
    return null;
}

private void saveData() {
    try (ObjectOutputStream oos1 = new ObjectOutputStream(new FileOutputStream("items.txt"));
        ObjectOutputStream oos2 = new ObjectOutputStream(new FileOutputStream("members.txt"))) {
        oos1.writeObject(items);
        oos2.writeObject(members);
    } catch (IOException e) {
        System.out.println("Error saving data.");
    }
}

@SuppressWarnings("unchecked")
private void loadData() {
    try (ObjectInputStream ois1 = new ObjectInputStream(new FileInputStream("items.txt"));
        ObjectInputStream ois2 = new ObjectInputStream(new FileInputStream("members.txt"))) {
        items = (List<LibraryItem>) ois1.readObject();
        members = (List<Member>) ois2.readObject();
        for (LibraryItem item : items) {
            usedIds.add(item.getId());
        }
        for (Member m : members) {
            usedIds.add(m.getId());
        }
    } catch (IOException | ClassNotFoundException e) {
        items = new ArrayList<>();
        members = new ArrayList<>();
    }
}

public static void log(String message) {
    try (FileWriter fw = new FileWriter("LibraryManagementSystem.txt", true)) {

```

```

        fw.write("[ " + LocalDateTime.now() + " ] " + message + "\n");
    } catch (IOException e) {
        System.out.println("Error writing to log file.");
    }
}
}

```

```

import java.io.*;
import java.time.LocalDate;

```

```

public abstract class LibraryItem implements Serializable {
    private static final long serialVersionUID = 1L;

```

```

    private final int id;
    private final String title;
    protected int quantity;
    private LocalDate borrowDate;
    private LocalDate dueDate;
    private boolean isLost;

```

```

    public LibraryItem(int id, String title, int quantity) {
        this.id = id;
        this.title = title;
        this.quantity = quantity;
        this.isLost = false;
    }

```

```

    public int getId() { return id; }
    public String getTitle() { return title; }
    public int getQuantity() { return quantity; }
    public boolean isLost() { return isLost; }
    public LocalDate getBorrowDate() { return borrowDate; }
    public LocalDate getDueDate() { return dueDate; }

```

```

    public boolean isAvailable() {
        return quantity > 0 && !isLost;
    }

```

```

    public void borrowItem() throws Exception {
        if (!isAvailable()) throw new Exception("Item not available or marked as lost.");
        quantity--;
        borrowDate = LocalDate.now();
        dueDate = borrowDate.plusDays(14);
    }

```

```

    public void returnItem() {
        quantity++;
    }

```

```

        if (LocalDate.now().isAfter(dueDate)) {
            isLost = true;
        }
        borrowDate = null;
        dueDate = null;
    }

    public abstract void displayDetails();

    public String logString() {
        return "ID=" + id + ", Title=" + title + ", Qty=" + quantity + (isLost ? " [LOST]" : "");
    }
}

class Book extends LibraryItem {
    public Book(int id, String title, int quantity) {
        super(id, title, quantity);
    }

    @Override
    public void displayDetails() {
        System.out.println("Book - " + logString());
    }
}

class Magazine extends LibraryItem {
    public Magazine(int id, String title, int quantity) {
        super(id, title, quantity);
    }

    @Override
    public void displayDetails() {
        System.out.println("Magazine - " + logString());
    }
}

import java.io.Serializable;
import java.time.LocalDate;

class Member implements Serializable {
    private static final long serialVersionUID = 2L;

    private final int id;
    private final String name;
    private final String password;
    private LibraryItem borrowedItem;

```

```

public Member(int id, String name, String password) {
    this.id = id;
    this.name = name;
    this.password = password;
}

public int getId() { return id; }
public String getName() { return name; }

public boolean authenticate(String input) {
    return password.equals(input);
}

public void borrowItem(LibraryItem item) throws Exception {
    if (borrowedItem != null) throw new Exception("You already borrowed an item.");
    item.borrowItem();
    borrowedItem = item;
    LibraryManagementSystem.log(name + " borrowed: " + item.logString() + " | Due: " +
item.getDueDate());
}

public void returnItem() throws Exception {
    if (borrowedItem == null) throw new Exception("You have not borrowed any item.");
    boolean isLate = LocalDate.now().isAfter(borrowedItem.getDueDate());
    borrowedItem.returnItem();
    if (isLate) {
        LibraryManagementSystem.log(name + " returned LATE: " + borrowedItem.logString() + " -
Marked LOST");
    } else {
        LibraryManagementSystem.log(name + " returned: " + borrowedItem.logString());
    }
    borrowedItem = null;
}

public void displayBorrowed() {
    if (borrowedItem != null) {
        System.out.println("Borrowed: Title = " + borrowedItem.getTitle() + ", Quantity = 1");
    } else {
        System.out.println("No borrowed item.");
    }
}
}

```